

REMOTE SYSTEM ADVANCED R&D — Minimum Remote Kernel v0.1

작성REMOTE SYSTEM ADVANCED R&D ENGINEER (Claude)

날짜2026-09-28

브랜치claude/remote-system-advanced-rd-5x2gsb · 보고서 커밋a8442e8

범위연구·설계와 실행 가능한 참조 구현. 9/7/4 Production, main, 2번 코드는 수정하지 않았다.

목차

0. 증거 등급 (먼저 읽을 것)
1. CURRENT REMOTE ARCHITECTURE ASSESSMENT
2. BOTTLENECK ROOT CAUSE
3. MINIMUM REMOTE KERNEL v0.1
4. BROWSER HARNESS RECOMMENDATION
5. SITE ADAPTER CONTRACT
6. DIGITAL TWIN / STEP MODEL
7. 9→7→4 PORTABILITY PLAN
8. SECURITY / AUTHORITY BOUNDARIES
9. CODE/FILE STRUCTURE PROPOSAL
10. WHAT NOT TO BUILD
11. FIRST IMPLEMENTABLE PATCH
12. HQ DECISION REQUIRED
- 부록 · 브라우저 증거 스크린샷

0. 증거 등급 (먼저 읽을 것)

등급	뜻	이 보고서에서 해당하는 것
VERIFIED	이 세션에서 직접 실행해 확인	research/remote-kernel-v0.1/ 참조 커널, node 테스트 21건, Playwright 브라우저 클릭 테스트 5건, 적합성 36/36, 변이 (mutation) 검사 8/8 검출
HQ-REPORTED / NOT_VERIFIED	HQ 지시문에 적혀 있으나 이 세션에서 코드·CI를 보지 못함	2번 gpt/remote-slice-001-vr, HEAD 81b96095..., Positive STEP PASS, Negative PUBLISH PASS, REAL_STATE_CHANGED=false, SIDE_EFFECT_COUNT=0
INFERRED	공개 동작이나 설계 추론	htmlpreview/Replit 실패 원인, 비용·한도 수치

2번 Remote Slice 소스는 이 세션에서 접근할 수 없었다. `list_repos` 결과는 `biztradepros/grok-knowledge-01` 하나뿐이며, 2번 저장소 이름도 지시문에 없다. 따라서 § 1의 2번 평가는 전부 NOT_VERIFIED이며, PASS로 올리지 않는다. 9/7/4 역시 실제 코드를 보지 않았고, 이 연구의 9/7/4는 **fixture(대역)**다. 실제 Factory 로직은 없다.

1. CURRENT REMOTE ARCHITECTURE ASSESSMENT

판정: **NOT_VERIFIED** (소스 미접근). HQ 보고 수치만으로 확정할 수 있는 것은 "VR 1개 Site에서 양성 1건과 음성 1건이 CI에서 초록"이라는 사실뿐이다.

HQ 보고만으로는 판단할 수 없고 코드에서 확인해야 하는 10개 항목이다. 각 항목은 § 3 적합성 검사 ID에 대응하므로, 2번에 shim만 붙여 돌리면 답이 기계적으로 나온다.

#	2번에서 확인할 질문	위험	대응 검사
Q1	<code>REAL_STATE_CHANGED=false</code> , <code>SIDE_EFFECT_COUNT=0</code> 이 측정값 인가, 리터럴 상수인가?	상수라면 CI PASS는 공허하다	C03/C04 (<code>real_state_probe: MEASURED</code>), F07 rogue 테스트
Q2	PUBLISH 거부가 커널/정책 층 에서 나는가, VR 코드 안의 if문인가?	Site마다 다시 구현해야 하고 빠뜨리기 쉽다	C08
Q3	STEP에 낙관적 잠금(<code>expect_seq</code>)이 있는가?	더블 클릭이나 두 HQ 클라이언트로 두 칸 전진	C05
Q4	idempotency key가 있고, 같은 키에 다른 payload가 오면 거부하는가?	재전송 시 중복 실행	C06, C07
Q5	STEP이 HOLD 를 반환할 수 있는가? (VR은 항상 전진일 가능성)	7번의 핵심 요구 <code>STEP ≠ ALWAYS ADVANCE</code> 불가	C03
Q6	결과 패킷이 hash chain 등 계보를 갖는가? replay가 가능한가?	증거를 사후에 재구성할 수 없음	C11
Q7	HQ 측 코드가 VR 엔진을 import하는가?	강결합, Site 로직 복제	<code>HQ client imports no site code</code> 테스트
Q8	끊김 후 재접속 시 같은 명령을 다시 보내면 재실행되는가?	Disconnect에서 중복 실행	C10
Q9	타임아웃 뒤 늦게 도착한 결과가 twin에 커밋될 수 있는가?	유령 전진	timeout 테스트
Q10	다른 site_id로 온 명령을 거부하는가?	오배송	C09

유지 원칙(지시 준수): 2번 구조는 그대로 둔다. 이 연구의 참조 커널은 교체안이 아니라 **계약(스키마) + 적합성 검사**다. 2번이 C01-C12를 통과하면 재작성은 0줄이다. 실패한 검사가 있으면 그 항목에 대한 delta patch만 제안한다(§ 11). "새 설계가 낫다"는 주장은 실패한 검사 ID로만 증명한다.

2. BOTTLENECK ROOT CAUSE

근본 원인(INFERRED): 방향이 거꾸로였다. 시도한 방법이 모두 "private 코드를 익명 공개 URL로 끌어내 브라우저가 가서 보는" 방식이었다. private repo에서는 이 방향이 원리적으로 막힌다.

시도	실패 메커니즘
htmlpreview.github.io	<code>raw.githubusercontent.com</code> 을 인증 없이 가져온다. private repo는 404가 되고, 설정으로 고칠 방법이 없다
Replit	private repo import에 별도 GitHub 연동이 필요하다. 성공하더라도 코드 사본과 preview URL이 repo 접근 경계 밖에 생긴다. 보안상 성공하면 안 되는 경로다

부차 원인: harness가 여러 파일로 된 앱(모듈 import, dev server)이라서 **파일 하나로 열 수 없었다**. `file://` 에서 ES module은 CORS로 막힌다.

해법의 방향: 브라우저를 코드가 있는 곳(CI runner, 로컬, 클라우드 에이전트 세션)으로 보내고, 밖으로는 증거(JSON·스크린샷)만 내보낸다. 앱은 공개하지 않는다.

3. MINIMUM REMOTE KERNEL v0.1

위치: `research/remote-kernel-v0.1/src/` · 의존성 0 · Node와 브라우저에서 같은 코드 · 핵심 669줄(kernel 309).



동사는 4개로 닫혀 있다: IDENTIFY (R0) · INSPECT (R0, view: twin|hold) · STEP (R1) · RESULT (R0).
EXPLAIN HOLD는 새 동사가 아니라 INSPECT {view:"hold"} 다. 동사를 늘리지 않기 위해서다 (§ 12 D4). 그 밖의

동사(PUBLISH 등)는 Site가 `actions`에 **선언만** 한다. v0.1 커널은 선언된 action을 절대 실행하지 않고, 정확한 사유로 거부하는 데만 쓴다.

Command Envelope (`schema/command.v0.1.schema.json`)

필드	필수	의미
<code>kind</code> / <code>protocol</code>	✓	<code>remote.command</code> / <code>remote.v0.1</code>
<code>command_id</code>	✓	RESULT 조회 키
<code>idempotency_key</code>	✓	같은 키와 같은 <code>command_hash</code> 이면 저장된 결과를 replay한다. 다른면 <code>IDEMPOTENCY_CONFLICT</code>
<code>site_id</code>	✓	불일치하면 <code>SITE_MISMATCH</code> . 전달(forward)하지 않는다
<code>session_id</code>	IDENTIFY 외 ✓	handshake 결과
<code>verb</code> , <code>args</code>	✓	
<code>expect_seq</code>	STEP ✓	twin 낙관적 잠금
<code>timeout_ms</code>		1-30000, Site 상한으로 clamp
<code>issued_by</code>	✓	HQ 운영자/에이전트 ID (계보)
<code>lineage.retry_of</code>		명시적 재발행만 가능하다(§ 6)

`command_hash` = hash(site_id, verb, args, expect_seq, issued_by). **session은 제외한다**. 그래야 재접속 후 새 세션으로 재전송해도 replay가 되고 재실행되지 않는다.

Result Envelope (`schema/result.v0.1.schema.json`) `status` ∈ `OK` | `HOLD` | `COMMAND_REJECTED` | `FAILED` | `TIMEOUT`, `reason_code` (20종), `hold.reasons[]`{code, detail, evidence_key}, `twin`{stage/seq/hash before·after}, `real_state_changed`, `real_state_probe`: `MEASURED` | `NOT_REACHED`, `side_effect_count` (v0.1에서 항상 0), `evidence_refs[]`, `lineage`{command_hash, issued_by, retry_of, parent_result_hash, prev_result_hash}, `result_hash`. `real_state_probe`가 이 설계의 핵심 추가다.

"false"가 측정한 값인지, 어댑터에 도달하지 않아 측정할 필요가 없었던 값인지를 패킷이 스스로 말한다(§ 1 Q1).

커널 불변식 (전부 테스트로 고정): I1 REJECTED/FAILED/TIMEOUT이면 twin hash가 변하지 않는다 · I2 HOLD는 stage를 바꾸지 않고 seq만 +1 · I3 ADVANCE는 정확히 다음 stage로만 간다(건너뛰기 불가) · I4 human-gate와 R2 전이는 adapter를 호출하기 **전에** 거부 · I5 같은 envelope는 최대 한 번만 실행 · I6 커널은 스스로 재시도하지 않는다 · I7 늦게 도착한 adapter 응답은 CAS로 커밋 불가 · I8 금지 키(prompt/model/secret/token...)는 커밋 전 차단 · I9 모든 결과(거부 포함)는 hash chain에 기록 · I10 `handle()`은 throw하지 않는다(실패는 패킷으로 반환).

VERIFIED : `npm test` 21/21. 핵심 가드를 하나씩 제거한 변이 8종을 모두 테스트가 잡았다(gate 잠금, human gate, replay, 실상태 probe, 누출 차단, 낙관적 잠금, hold 해제, NO_SESSION 키 소모). 처음 돌렸을 때

NO_SESSION 번이 1종이 살아남아 테스트를 추가했다.

4. BROWSER HARNESS RECOMMENDATION

방식	보안	구현량	private 호환	재현성	자동 Browser Test	9/7/4 재사용
① local/static test server	좋음 (localhost)	작음	✓	중 (로컬 환경 의존)	✓	✓
② authenticated preview (Vercel/Netlify 등 보호 배포)	중 (외부 호스트에 사본, 토큰 관리)	중-큼	조건부 (유료·연동)	중	△ (인증 우회 토큰 필요)	△
③ CI 임시 preview 배포	약-중 (짧게라도 외부 노출)	큼	조건부	중	✓	△
④ bundled standalone 단일 HTML	매우 좋음 (CSP로 네트워크 0)	작음	✓ (파일 하나)	높음	✓ (file://)	✓
⑤ Actions artifact + browser harness	매우 좋음 (content:read, 시크릿 0)	작음	✓ (artifact는 repo 읽기 권한자만)	높음 (pinned)	✓	✓
⑥ DREAM CONTROL bounded test route	좋음 (기존 인증 뒤)	중	✓	중	✓	✓

추천: ④ + ⑤ 조합, 하나의 아키텍처. 빌드가 커널·어댑터·적합성 검사를 단일 HTML (dist/remote-harness.html, 63 KiB, CSP default-src 'none')로 묶는다. 이어서 GitHub Actions에서 pinned Playwright Chromium이 file:// 로 열어 실제 클릭을 수행한다. evidence/ (summary.json, ledger 전체, 스크린샷)와 HTML 자체는 14일 보존 artifact로 올린다. 공개 URL, 외부 호스트, 시크릿은 없다.

- ①은 같은 테스트의 개발용 fallback이다 (HARNESS_URL=http://127.0.0.1:...). 추가 코드는 0줄이다.
- ⑥은 나중에 DREAM CONTROL이 인증된 경로를 가지면 HARNESS_URL 만 바꿔 같은 테스트를 재사용한다.
- ②③은 권하지 않는다. private 코드를 외부 호스트에 두는 것 자체가 이번 병목을 만든 방향이다.

VERIFIED (이 세션, Chromium 141): `file://` 모드 5/5 PASS, localhost 모드 5/5 PASS. off-origin 요청 0, console error 0. `summary.json`: `POSITIVE_STEP=PASS, NEGATIVE_PUBLISH=PASS, REAL_STATE_CHANGED=false, SIDE_EFFECT_COUNT=0, CONFORMANCE=36/36, NETWORK_ISOLATION=PASS`.

NOT_VERIFIED: `ci/remote-browser-harness.yml` 은 템플릿이라 실제 GitHub Actions에서는 아직 돌려보지 않았다. private repo의 Actions 분·artifact 한도는 현재 GitHub 가격 페이지로 재확인해야 한다.

5. SITE ADAPTER CONTRACT

어댑터는 **Site 저장소 안에** 있고 Site가 소유한다. HQ는 어댑터를 import하지 않는다(테스트로 강제).

```
/** @type {SiteAdapter} */
{
  adapter_id, adapter_version,
  identity: { site_id, site_name, factory_no },
  pipeline: { id, stages: [...선행], gates?: { [toStage]: { authority: 'R1'|'R2', human_gate?: true } } },
  actions?: { [VERB]: { authority, human_gate?, external_effect?, description } }, // 선언 = 거부용
  genesis_stage?, genesis_facts?, max_authority?: 'R0'|'R1', max_timeout_ms?,
  async step({ twin /*deep-frozen*/, from, to, args, effects /*모든 attempt 거부*/ })
    → { decision: 'ADVANCE', to, evidence_refs?, facts? } | { decision: 'HOLD', reasons: [{code, detail, evidence_key?}] },
  project?(twin) → 공개 view,
  realFingerprint() → 실상태의 읽기 전용 hash
}
```

규칙:

- 어댑터가 받는 것은 동결된 twin, 읽기 전용 engine port, 그리고 모든 시도를 거부하는 `effects` 싱크뿐이다. **실상태 write port는 인터페이스에 존재하지 않는다.**
- 어댑터는 **제안만** 한다. twin을 바꾸는 것은 커널의 순수 함수 `applyDecision` 뿐이다. 그래서 replay는 엔진 없이도 결정적이다.
- 공통 STEP을 Site 의미로 **번역**하는 것이 어댑터의 유일한 일이다. 예를 들어 7번에서 STEP은 "M1로 가도 되는지 refinery gate에 물어보기"가 된다.
- 한계를 정직하게 적는다. JS 어댑터는 원하면 부수 경로로 실상태를 건드릴 수 있다. 커널은 `realFingerprint` 가 덮는 범위의 변화만 **탐지**(FAILED `REAL_STATE_CHANGED`, `real_state_changed: true`로 정직하게 보고)하며 **방지**하지는 못한다. 따라서 fingerprint 범위는 Site 소유자가 정하고 리뷰한다.

6. DIGITAL TWIN / STEP MODEL

```
TwinState { site_id, pipeline_id, stage, seq, hold|null, evidence_refs[], facts{stage→public facts} }
```

```
STEP(expect_seq)
```

```
└ expect_seq ≠ seq → COMMAND_REJECTED STALE_TWIN (twin 불변)
└ stage = 마지막 → COMMAND_REJECTED TERMINAL_STAGE
└ gates[next] human_gate / R2 / 권한 부족 → COMMAND_REJECTED HUMAN_GATE_REQUIRED |
R2_DISABLED | AUTHORITY_INSUFFICIENT
└ (adapter 미호출, real_state_probe = NOT_REACHED)
└ probe1 → adapter.step (timeout) → probe2
└ └ timeout → TIMEOUT (twin 불변, 늦은 응답은 CAS로 폐기)
└ └ throw / 동결 twin 변경 → FAILED ADAPTER_ERROR
└ └ probe1 ≠ probe2 → FAILED REAL_STATE_CHANGED (true로 보고)
└ └ effects.attempt() → FAILED SIDE_EFFECT_ATTEMPTED (삼켜도 탐지)
└ └ 잘못된 decision / 단계 건너뛰기 → FAILED ILLEGAL_TRANSITION
└ └ 금지 키 포함 → FAILED LEAK_BLOCKED
└ ADVANCE → stage = next, seq+1, hold = null → OK
└ HOLD → stage 동일, seq+1, hold = {reasons, at_seq} → HOLD
```

HOLD와 REJECT의 의미 구분

	HOLD	COMMAND_REJECTED
명령이 유효했나	예	아니오 (권한·형식·잠금·gate)
Site가 판단했나	예 (Site gate가 "아직 아님")	아니오 (커널이 adapter 이전에 차단)
twin	seq+1, hold 기록 (증거가 됨)	불변
다음 행동	Site 쪽 작업 후 새 <code>expect_seq</code> 로 STEP	원인 해소 후 새 키로 재발행
설명	<code>INSPECT{view:"hold"}</code> = EXPLAIN_HOLD (기록된 사유를 그대로 반환, 재계산하지 않음)	<code>reason_code</code> + <code>data</code>

재시도 규칙:

- 커널은 절대 재시도하지 않는다.
- HQ는 같은 **envelope 재전달**(`redeiver`)만 자유롭게 할 수 있다. 결과는 replay이고 실행은 최대 1회다.
- TIMEOUT이나 FAILED 후 같은 의도를 다시 실행하려면 새 키 + `lineage.retry_of`가 필요하다(`reissue`).
이로써 자동 재시도는 구조적으로 불가능해지고, 모든 재시도가 계보에 남는다.
- 세션 없음(NO_SESSION)이나 세션 만료(SESSION_EXPIRED)로 인한 거부는 키를 소모하지 않는다. 그래야 재접속 후 같은 명령이 정상 처리된다.

Disconnect Survival: 커널과 twin, ledger는 Site 안에 있다. HQ가 끊겨도 Site는 계속 동작하고 결과를 기록한다 (ALONE = WORKS). 재접속하면 IDENTIFY로 새 세션을 받고, `RESULT(command_id)` 나 `redeliver`로 결과를 회수한다(DISCONNECTED = SURVIVES). 테스트 C10과 "HQ disconnect after delivery"가 이를 확인한다.

Replay / Lineage: `verifyChain(ledger)`는 모든 `result_hash`를 재계산하고 `prev_result_hash` 연결을 검사한다. `replayTwin(genesis, ledger)`는 기록된 decision만으로 twin을 재구성해 각 단계의 hash_before/after와 대조한다. 9번에서는 엔진 artifact를 **삭제한 뒤에도** replay hash가 일치함을 확인했다(**VERIFIED**).

7. 9→7→4 PORTABILITY PLAN

	9 SKILL/AI	7 DATA REFINERY	4 SOCIAL CARD
stages	SOURCE→RAW→KNOWL EDGE→REFINEMENT→C ANDIDATE→EVIDENCE	SOURCE→M0→M1→M2→M2_V ERIFIED→F1_HANDOFF	SOURCE→RECIPE→ARTIFA CT→QC→HANDOFF→(PUB LISHED)
검증 대상	비결정 산출물과 결정적 전이의 분리	<code>STEP ≠ ALWAYS ADVANCE</code> , <code>HOLD/EXPLAIN_HOLD</code>	Human Gate 경계
HOLD 코드	<code>ARTIFACT_NOT_READY</code> , <code>EVAL_FAILED</code>	<code>MISSING_EVIDENCE{evidence_ke y}</code> , <code>VALUE_GATE{detail}</code>	<code>MISSING_INPUT</code> , <code>QC_FAILED{evidence_key}</code>
R2 action (선언 → 거부)	<code>RUN_MODEL</code> → <code>R2_DISABLED</code>	<code>DELIVER_F1</code> → <code>R2_DISABLED</code>	<code>PUBLISH</code> → HUMAN_GATE_REQUIRED (gate와 action 이중 잠금)
Site가 노출할 읽기 port	<code>sealedArtifact(stage)</code> → <code>{artifact_ref, eval{passed, band}}</code>	<code>gate(to) → {pass, missing[], value_gate{pass, reason}, evidence_refs}</code>	<code>evaluate(to) → {ok, facts, refs} \ {reasons}</code>
HQ에 가지 않는 것	prompt, model, 원문 output (금지 키로 차단 + 테스트)	M0/M1/M2 계산식, Value Gate 임계값(테스트로 확인: <code>0.7</code> , <code>0.02</code> 가 패킷에 없음)	게시 계정, 게시 코드 (어댑터에 publisher가 없음)

9 설계 요점:

- STEP은 모델을 **호출하지 않는다**. 모델 호출은 provider execution, 즉 R2다.
- Site의 AI 작업은 Site 일정대로 돌고, 산출물은 `artifact_ref` (hash)와 eval 결과로 봉인된다. STEP은 "다음 단계용 봉인 artifact가 있고 eval을 통과했는가"만 묻는다.
- 비결정성은 artifact 안에 갇힌다. twin 전이는 ref만 기록하는 결정적 함수다.

7 설계 요점:

- HQ는 M0/M1/M2 논리를 소유하지 않는다. 어댑터는 `engine.gate(to)`를 번역할 뿐이다.

- 증거가 부족하면 STEP이 HOLD로 끝나고, EXPLAIN_HOLD가 누락 증거 키와 Value Gate 사유 문장을 반환한다. 임계값 수치는 반환하지 않는다.
- `genesis_stage`로 실제 현재 단계(예: M2)에서 twin을 분기할 수 있다.
- F1_HANDOFF는 twin 안에서 "handoff packet 준비"까지다. 실제 전달은 R2 `DELIVER_F1`이다.

4 설계 요약:

- HANDOFF까지는 R1 시뮬레이션이다. 렌더는 로컬 결정적 spec hash이며 업로드하지 않는다.
- PUBLISH는 두 경로 모두 `COMMAND_REJECTED / HUMAN_GATE_REQUIRED`로 끝난다: HANDOFF에서의 STEP, 그리고 명시적 PUBLISH 동작.
- 이때 `real_state_probe = NOT_REACHED`다. 어댑터에게 묻지조차 않는다. 실상태 스냅샷도 불변이다.

단계별 진입 조건(각 Factory 공통):

1. 해당 Factory repo에 `remote/` 추가(§ 9)
2. 실제 engine의 읽기 port를 감싸는 어댑터 작성
3. fixture가 아닌 실제 어댑터로 C01-C12 PASS
4. harness workflow에서 artifact 확인
5. HQ 승인 후 다음 Factory

9 → 7 → 4 순서는 HQ 지시를 따른 것이다. 9에서 계약의 가장 약한 고리(비결정성과 누출)를 먼저 깨뜨려 보자는 의미로도 합리적이다.

8. SECURITY / AUTHORITY BOUNDARIES

권한	허용	v0.1 상태	강제 지점
R0	IDENTIFY, INSPECT, RESULT (읽기)	활성	커널 <code>VERB_AUTHORITY</code>
R1	STEP (twin만 변경)	활성, Site 최대	세션 <code>granted = min(요청, site max, KERNEL_MAX=R1)</code>
R2	실상태·외부 효과 (게시, 결제, provider 실행, 자격증명, 실제 handoff)	코드상 비활성	<code>KERNEL_MAX_AUTHORITY='R1'</code> . 요청해도 R1로 깎이고 <code>note: R2_DISABLED</code>
Human Gate	PUBLISH 등	원격으로는 어떤 권한으로도 통과 불가	권한 검사보다 먼저 판정

추가 경계:

- 금지 키 누출 차단(커밋 전과 응답 직전, 두 번)

- 부수효과 싱크(시도가 있으면 삼켜도 FAILED)
- 실상태 fingerprint
- 동결 twin
- harness CSP(네트워크 0)
- CI `permissions: contents: read`, `persist-credentials: false`, 시크릿 0

v0.1이 다루지 않는 위협(명시):

1. 전송 인증: session_id는 bearer 문자열이다. 현재는 in-process 전송만 가정한다. 네트워크 전송 전에 mTLS나 서명된 envelope가 필요하다.
2. 영속화: ledger가 메모리에 있다. Site별 append-only 파일/DB는 Site 소유 과제다.
3. 악성 어댑터: 탐지만 할 수 있고 방지는 못한다(§ 5-4).
4. R2 설계: 사람 승인 토큰, 2인 승인 등은 **의도적으로 미설계**이며 별도 HQ 결정 대상이다.

9. CODE/FILE STRUCTURE PROPOSAL

이 연구에서 만든 것(현재 브랜치):

```
research/remote-kernel-v0.1/
  REPORT.md                이 문서
  package.json              의존성 0 (Playwright는 CI에서만 --no-save)
  schema/command.v0.1.schema.json · result.v0.1.schema.json  규범 계약 (테스트로 상수와 동기화)
  src/constants.js sha256.js canonical.js envelope.js twin-store.js kernel.js  커널 (669줄)
  src/conformance.js        C01-C12, Site-neutral
  src/hq-client.js          HQ 측: envelope + transport만
  adapters/*.fixture.js index.js  9/7/4 대역 (실제 로직 없음)
  harness/build.js harness.template.html  단일 HTML 번들러
  test/kernel.test.js conformance.test.js  node:test 21건
  test/browser/harness.browser.js          Playwright 클릭 + evidence/ 기록
  ci/remote-browser-harness.yml            비활성 템플릿
```

Factory 저장소에 적용할 때(제안, HQ 승인 후):

```
<factory-repo>/
  remote/
    kernel/          ← 위 src/ 를 버전 고정 사본으로 vendoring (+ KERNEL.sha256)
    adapter.js       ← Site 소유. 기존 engine의 읽기 port만 import
    fixtures.js      ← prepareAdvance: 테스트용 Site 쪽 준비
    conformance.test.js
    .github/workflows/remote-browser-harness.yml
  HQ (DREAM REMOTE):
    hq-client.js + schema/ ← Site 코드 import 금지
```

npm 패키지 대신 **vendoring**을 택한다. private 레지스트리 없이도 동작하고(ALONE = WORKS), Site가 커널 버전을 스스로 고정하며, HQ가 끊겨도 빌드가 된다. `KERNEL.sha256`으로 사본 변조를 검사한다.

10. WHAT NOT TO BUILD

- 범용 워크플로/그래프 엔진. v0.1은 **선형 stage**만 다룬다. 9/7/4 모두 선형이다. 분기(QC 실패 시 되돌림)는 HOLD로 충분하다.
 - 새 동사. EXPLAIN_HOLD, PREVIEW 등은 INSPECT view나 선언된 action으로 처리한다.
 - HQ 쪽 Site 로직 사본, HQ 중앙 twin DB. twin은 Site가 소유한다.
 - R2 실행 경로, 사람 승인 UI, 자동 게시, 결제, provider 호출.
 - 커널의 자동 재시도나 백오프.
 - 외부 preview 호스팅(Replit, htmlpreview, Vercel preview 등)과 공개 데모 URL.
 - 메시지 브로커, WebSocket 서버, 인증 서버. 전송은 다음 단계에서 결정한다.
 - 2번 VR 코드 재작성. 적합성 검사가 실패를 증명하기 전에는 하지 않는다.
 - JSON Schema 런타임 검증 라이브러리(ajv 등). 손으로 쓴 검증기와 스키마 동기화 테스트로 충분하다.
-

11. FIRST IMPLEMENTABLE PATCH

PATCH-0 "2번 적합성 측정" — 2번 로직 변경 0줄, 2번 repo의 새 연구 브랜치(main 아님)에 적용한다.

1. `remote/kernel/`에 이 연구의 `src/`를 사본으로 넣는다(적합성 검사 실행용).
2. `remote/vr-shim.js`에서 2번의 기존 명령 처리기를 `{ handle(cmd) → result }` 모양으로 감싼다. 2번 API를 보지 못했으므로 아래는 **가정**이다. 실제 함수명에 맞춰야 한다.

```
// ASSUMPTION: 2번에 handleRemoteCommand(input) → output 형태의 진입점이 있다.
import { handleRemoteCommand } from '../src/remote/...'; // 실제 경로로 교체
export const makeShimKernel = () => ({
  async handle(cmd) { const out = await handleRemoteCommand(toVrInput(cmd)); return
    toResultEnvelope(cmd, out); },
  // twin(), genesis(), ledger(): 2번이 제공하지 못하면 해당 검사는 FAIL로 남긴다(PASS로 위장 금지)
});
```

3. `remote/conformance.test.js`에서 `runConformance({ makeSite, prepareAdvance, makeKernel: makeShimKernel })`를 실행한다.
4. `ci/remote-browser-harness.yml`을 2번 `.github/workflows/`에 넣는다. private repo 안에서 브라우저 클릭 경로를 확보하고, **Browser Infra HOLD**를 해제한다.
5. 산출물: C01-C12 표(PASS/FAIL), `evidence/summary.json`. FAIL 항목마다 delta patch를 1개씩 따로 제안한다(예: Q1 실패 시 실상태 probe 추가, Q3 실패 시 `expect_seq` 추가).

규모 추정(**INFERRED**): shim과 매핑 100-200줄, workflow 1개. 2번 동작 코드는 변경하지 않는다.

이 브랜치에서 지금 바로 재현하는 방법:

```
cd research/remote-kernel-v0.1
npm test # 21 pass
npm run build:harness # dist/remote-harness.html
npm i --no-save playwright@1.56.1 && npx playwright install chromium
npm run test:browser # 5 pass, evidence/ 생성
```

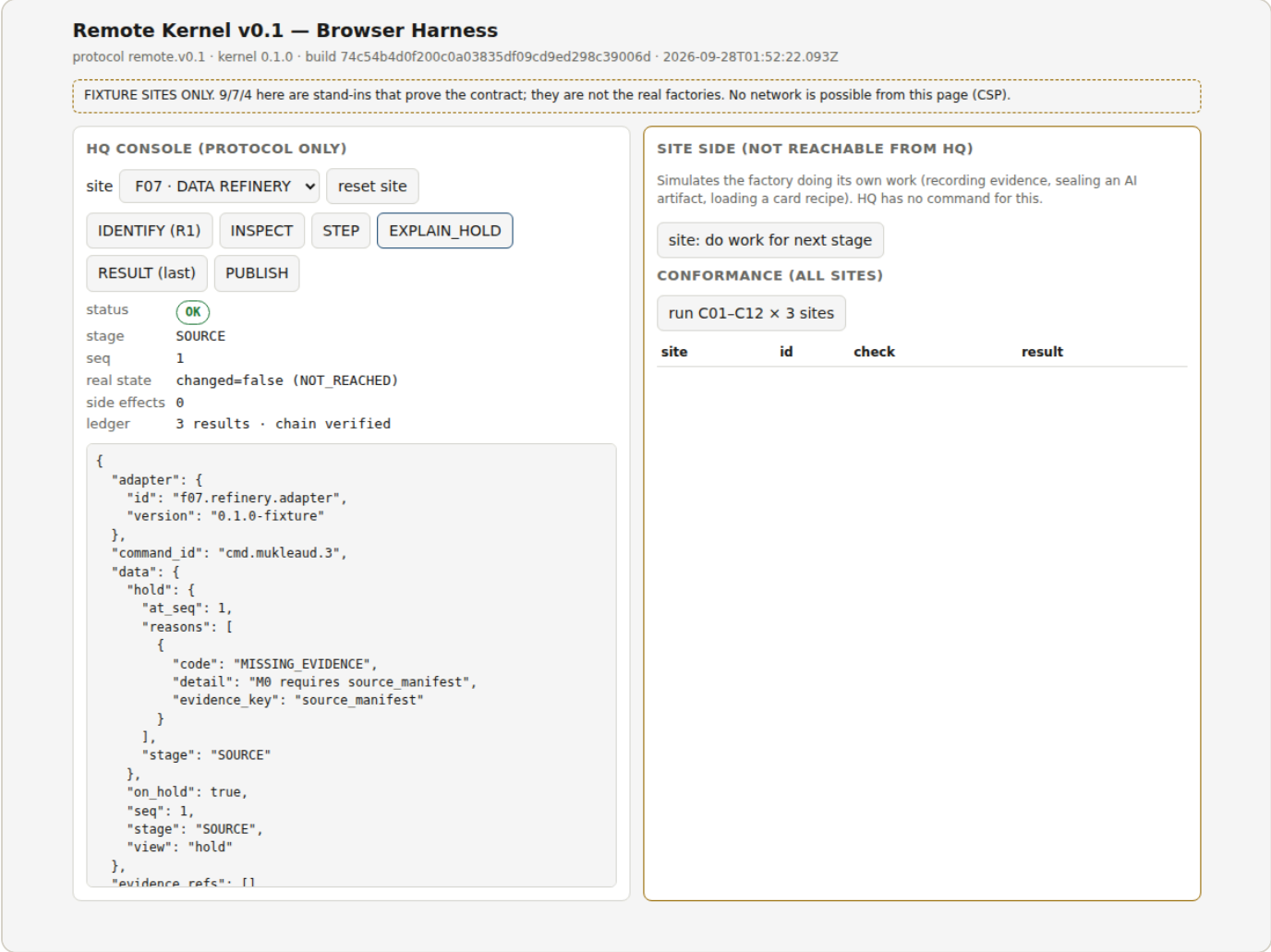
12. HQ DECISION REQUIRED

ID	결정 사항	추천
D1	이 R&D 라인에 2번 저장소 읽기 권한 부여 (PATCH-0 실행과 § 1 Q1-Q10 판정에 필요)	부여. 없으면 § 1은 계속 NOT_VERIFIED
D2	2번 유지 여부 판정 규칙: "C01-C12 실패 항목만 delta patch, 통과 시 재작성 0"	승인
D3	2번(이후 9/7/4) private repo에 GitHub Actions 분·artifact 사용	승인. job당 10분 상한, 보존 14일
D4	EXPLAIN_HOLD = INSPECT{view:"hold"} (새 동사 아님)	승인. 동사 4개 유지
D5	HOLD가 seq 를 +1 한다(HOLD도 twin 이벤트로 기록)	승인. 대안은 HOLD를 ledger에만 남기는 것
D6	R2는 v0.1에서 코드상 비활성으로 유지하고, 사람 승인 설계는 별도 과제로	승인
D7	커널 배포 방식: Factory별 vendoring 사본 + sha256	승인
D8	이 연구물은 public 저장소 biztradepros/grok-knowledge-01 의 브랜치에 push 되어 있다. 비밀이나 단가는 없지만 Factory 구조 설계가 공개 상태다	private 저장소로 옮길지 결정. 옮기면 이 브랜치를 삭제
D9	9→7→4 각 단계의 진입 승인자	HQ 지정

STOP. 9/7/4 Production, main, 2번 코드는 수정하지 않았다. 이 문서와 참조 구현을 HQ로 반환한다.

부록 · 브라우저 증거 스크린샷

이 세션의 Playwright Chromium 실행에서 저장한 화면이다. 대상은 fixture Site이며 실제 Factory가 아니다.



F07 DATA REFINERY — STEP → HOLD, EXPLAIN_HOLD가 누락 증거 source_manifest를 반환

Remote Kernel v0.1 — Browser Harness

protocol remote.v0.1 · kernel 0.1.0 · build 74c54b4d0f200c0a03835df09cd9ed298c39006d · 2026-09-28T01:52:22.093Z

FIXTURE SITES ONLY. 9/7/4 here are stand-ins that prove the contract; they are not the real factories. No network is possible from this page (CSP).

HQ CONSOLE (PROTOCOL ONLY)

site **F04 · SOCIAL CARD** ▼

reset site

IDENTIFY (R1)

INSPECT

STEP

EXPLAIN_HOLD

RESULT (last)

PUBLISH

status **COMMAND_REJECTED** HUMAN_GATE_REQUIRED

stage HANDOFF

seq 4

real state changed=false (NOT_REACHED)

side effects 0

ledger 7 results · chain verified

```
{
  "adapter": {
    "id": "f04.social-card.adapter",
    "version": "0.1.0-fixture"
  },
  "command_id": "cmd.mukleb9n.11",
  "data": {
    "granted": "R1",
    "human_gate": true,
    "required_authority": "R2",
    "verb": "PUBLISH"
  },
  "evidence_refs": [],
  "hold": null,
  "idempotency_key": "idem.cmd.mukleb9n.11",
  "kernel_version": "0.1.0",
  "kind": "remote.result",
  "lineage": {
    "command_hash": "sha256:d4e6d89bbe2e574cbd64300bd9f5b084c980df2c",
    "issued_by": "hq.harness",
    "parent_result_hash": null,
    "prev_result_hash": "sha256:8440a773d027f41735f2c630cc4e36d17058",
    "retry_of": null
  }
}
```

SITE SIDE (NOT REACHABLE FROM HQ)

Simulates the factory doing its own work (recording evidence, sealing an AI artifact, loading a card recipe). HQ has no command for this.

site: do work for next stage

CONFORMANCE (ALL SITES)

run C01-C12 × 3 sites

site	id	check	result
------	----	-------	--------

F04 SOCIAL CARD — HANDOFF 이후 PUBLISH → COMMAND_REJECTED / HUMAN_GATE_REQUIRED, real_state_changed=false, side_effect_count=0

원본: [research/remote-kernel-v0.1/REPORT.md](#) · 이 파일은 네트워크 없이 열린다(글꼴·이미지 내장).